Temporaalitietokannat

Ajassa muuttuva data

Oppimistavoitteet

1. Ymmärretään ajassa tapahtuvien muutosten mallintamisen tarve ja ajan ulottuvuudet tietokannassa.
2. Opitaan luomaan temporaalitietokanta käyttäen tiedonhallintaohjelmiston tarjoamia työkaluja.

Lähtökohtia

- Aiemmin on opittu loogista tietokantasuunnittelua eli tietokannan skeeman suunnittelua.
- Reaalimaailmassa asiat tapahtuvat ajan kuluessa, ja sen seurauksena data muuttuu.
- Datan muuttuessa syntyy historiaa, jonka tallentaminen voi olla tärkeää.
  - Mitä aikaisempia arvoja tietoalkiolla on ollut?
  - Millä hetkellä kukin arvon muutos on tapahtunut?
  - Milloin kukin arvo on ollut voimassa?
- Lähes kaikkiin tietokantoihin sisältyy jossain määrin ajallista ulottuvuutta.
- Sen mallintaminen tauluin ja kentin skeemassa voi olla haastavaa.
- Tiedonhallintaohjelmistot tarjoavat tukea ajan mallintamiseen.

Tietokanta ilman aikaulottuvuutta

id	firstname	lastname	salary
1	Viivi	Virta	2 000,00
2	Ahmed	Bakir	2 000,00

- Esimerkissä on yksinkertainen Työntekijä-taulu.
- Oletetaan, että Viivi saa palkankorotuksen 2200 euroon.
- Yksinkertainen ratkaisu: vaihdetaan salary-kentän arvo.
- Palkkojen historia ei kuitenkaan tallennu. Esimerkiksi seuraavat haasteet syntyvät:
  1. Mitä tehdään, jos palkkojen historiaa on tarkasteltava myöhemmin?
     - Miten esim. 12.3.2022 tietokannassa ollut palkka voitaisiin hakea?
  2. Mitä tehdään, jos Viivin palkka halutaan korottaa 2500 euroon kuun vaihteessa noin kahden kuukauden päästä, kun koeaika päättyy?
     - Voisiko palkan päivittää jo etukäteen olemaan voimassa tietystä päivästä lukien?

Temporaalitietokanta

- Edellä kuvatun kaltaisia ongelmia vältetään ottamalla käyttöön temporaalitietokanta (*temporal database*).
  - *temporal* = aikaan liittyvä.
- Temporaalitietokannoissa puhutaan kolmesta aikaulottuvuudesta:
  1. Kirjausajasta (*transaction time*)
  2. Kelpoisuusajasta (*valid time*)
  3. Päätösajasta (*decision time*)
- Näistä voidaan ottaa käyttöön yksi tai useampi ulottuvuus.
- Yleensä käytetään kirjaus- ja kelpoisuusaikaa tai jompaakumpaa niistä.

Ulottuvuus 1/3: kirjausaika

id	firstname	lastname	salary	transaction_time
1	Viivi	Virta	2 000,00	2022-08-12 10:14:41.649413
1	Viivi	Virta	2 300,00	2022-08-12 10:14:44.708788
2	Ahmed	Bakir	2 000,00	2022-08-12 10:14:41.654144

- Kirjausaika (*transaction time*) kertoo
  - ajan hetken, jolla tieto on kirjattu tietokantaan, tai
  - aikavälin, jolloin tietue on ollut ajantasainen eli viimeksi kirjattu tietue.
- Huomaa, että Viivin palkkaa on korjattu kolme sekuntia alkuperäisen palkan syötön jälkeen.
- Tietokantataulussa on nyt kirjausajan aikaulottuvuus mukana.
- Historialliset arvot ja arvojen muutosajankohdat tallentuvat.
- Kirjausaika ei välttämättä ole aika, jolloin muuttunut tieto on sovelluksen näkökulmasta voimassa.
  - Jos Viivi aloitti työt elokuun alussa., saa hän todennäköisesti tuon päivän aamusta lukien palkkaa 2300€/kk eikä aluksi ole tarkoitus maksaa alhaisempaa palkkaa.
- Riittääkö kirjausajan tallentaminen palkkahistorian tallentamiseksi?
  - Kirjausaika ehkä riittää, jos pidetään huoli, että aina palkanmaksupäivien (esim. kuun viimeinen päivä) välissä ao. kuun palkka on päivitetty, ja palkka koskee aina koko kuukautta.
  - Tulevia palkankorotuksia ei voi syöttää esimerkiksi kahta kuukautta etukäteen.

Ulottuvuus 2/3: kelpoisuusaika

id	firstname	lastname	salary	startdate	enddate
1	Viivi	Virta	2 500,00	2022-11-01	2024-12-31
2	Ahmed	Bakir	2 000,00	2022-08-01	2024-12-31
3	Viivi	Virta	2 000,00	2022-08-01	2022-11-01

- Kelpoisuusaika (*valid time* / *application time*) kertoo, milloin tieto on voimassa sen käytön näkökulmasta.
- Se ei ota kantaa siihen, milloin tieto on tallennettu tietokantaan.

Ulottuvuus 3/3: päätösaika

id	firstname	lastname	salary	decision_time
1	Viivi	Virta	2 000,00	2022-07-24
2	Ahmed	Bakir	2 000,00	2022-07-31

- Päätösaika (*decision time*) sisältää tiedon siitä, milloin tietueen kelpoisuutta koskeva päätös on tehty.
  - Voi poiketa sekä kirjausajasta että kelpoisuusajasta.
- Esimerkki päätös-, kelpoisuus- ja kirjausajasta: 24.7.2022 pidetyssä kokouksessa päätettiin, että Viivin palkka välillä 1.8.2022-31.12.2022 on 2000 €/kk. Tieto syötettiin järjestelmään 27.7.2022.
- Päätösaika on tietokannan näkökulmasta yleensä tavallinen päivämäärä- tai aikakenttä, eikä se ole erityisen mielenkiintoinen.

SQL ja aikaulottuvuudet

- SQL:2011-standardi määrittää operaatiot bitemporaaliselle tietokannalle.
  - Bitemporaalinen tietokanta yhdistää kelpoisuusajan ja kirjausajan.
- Esimerkiksi MariaDB:ssä on versiosta 10.3.4 ollut tuki bitemporaalisille tietokannoille.
  - *system versioning*
  - *application-time period versioning*
- Toiminnallisuus täydentyy ajan myötä: esimerkiksi versiosta 10.5.3 alkaen mukana on WITHOUT OVERLAPS -määreen tuki, jolla voidaan estää päällekkäiset aikavälit.
  - tapausesimerkkinä hotellihuoneeseen kohdistuvat varaukset, joita saa olla yksi per ajan hetki
- Tarkastellaan seuraavaksi erikseen kummankin aikaulottuvuuden käyttöä MariaDB-ohjelmistossa.

Kirjausaikaulottuvuuden käyttöönotto

```
CREATE TABLE person (
    id INT NOT NULL AUTO_INCREMENT,
    firstname VARCHAR(40),
    lastname VARCHAR(40),
    salary DECIMAL(8,2),
    PRIMARY KEY (id)
) WITH SYSTEM VERSIONING;
```

- Lopussa oleva määre WITH SYSTEM VERSIONING kytkee kirjausaikaan perustuvan aikaversioinnin päälle.
- Tauluun tulee tällöin piilotetut erikoiskentät row_start ja row_end, johon aikaleimat tallentuvat.
  - Ne eivät oletusarvoisesti näy.

Kyselyt kirjausajan mukaan

```
INSERT INTO person VALUES (NULL, "Viivi", "Virta", 2000);
INSERT INTO person VALUES (NULL, "Ahmed", "Bakir", 2000);
```

1.
```
SELECT * FROM person;

SELECT SLEEP(3);
UPDATE person SET salary=2300 WHERE id=1;
```

2.
```
SELECT * FROM person;
```

3.
```
SELECT *, row_start AS transaction_time FROM person FOR SYSTEM_TIME ALL;
```

4.
```
SELECT * FROM person FOR SYSTEM_TIME AS OF TIMESTAMP SUBTIME(NOW(), "1.0");
```

1.

id	firstname	lastname	salary
1	Viivi	Virta	2 000,00
2	Ahmed	Bakir	2 000,00

2.

id	firstname	lastname	salary
1	Viivi	Virta	2 300,00
2	Ahmed	Bakir	2 000,00

3.

id	firstname	lastname	salary	transaction_time
1	Viivi	Virta	2 000,00	2022-08-12 12:51:32.725143
1	Viivi	Virta	2 300,00	2022-08-12 12:51:35.780547
2	Ahmed	Bakir	2 000,00	2022-08-12 12:51:32.729443

4.

id	firstname	lastname	salary
1	Viivi	Virta	2 000,00
2	Ahmed	Bakir	2 000,00

Kyselyt kirjausajan mukaan

- Kirjausajan mukaan versioiduissa tauluissa tietokantaoperaatiot (SELECT, INSERT, UPDATE, DELETE) voidaan kirjoittaa tavalliseen tapaan kuin versiointia ei olisikaan.
- Tavallinen SELECT palauttaa ajantasaisen tietueen.
- Tavallinen DELETE merkitsee tietueen ajantasaisen version vanhentuneeksi, mutta ei tuhoa tietuetta fyysisesti.
  - Tietueen kaikki versiot voidaan tarvittaessa tuhota DELETE HISTORY -lauseella.

Kyselyt kirjausajan mukaan

- Historiaversioon voidaan viitata lisäämällä loppuun FOR SYSTEM_TIME -määre.
- Esimerkkejä:
  - SELECT ... FOR SYSTEM_TIME ALL hakee kaikki aikaversiot erikseen
  - SELECT ... FOR SYSTEM TIME SUBTIME(NOW(), ”365 0:0:0”) hakee 365 vuorokauden takaisen version;
  - SELECT ... FOR SYSTEM TIME TIMESTAMP("2022-08-24 18:55") hakee 24.8.2022 klo 18:55 voimassa olleen version.

Kelpoisuusaikaulottuvuuden käyttöönotto

```
CREATE TABLE person (
   id INT NOT NULL AUTO_INCREMENT,
   firstname VARCHAR(40),
   lastname VARCHAR(40),
   salary DECIMAL(8,2),
   startdate DATE,
   enddate DATE,
   PERIOD FOR valid_time(startdate, enddate),
   PRIMARY KEY (id)
);
```

- Taulukkoon luodaan kentät kelpoisuusajankohdan alku- ja loppuajoille.
  - Sallittuja tietotyyppejä ovat DATE, DATETIME ja TIMESTAMP, ja niiden on oltava keskenään samat.
- PERIOD FOR -määre kytkee kelpoisuusaikaan perustuvan versioinnin päälle.
  - Esimerkissä `valid_time` on itse määrätty tunniste, jolla kelpoisuusaikaväliin voidaan myöhemmin viitata.

Kyselyt kelpoisuusajan mukaan

```
INSERT INTO person VALUES (NULL, "Viivi", "Virta", 2000, "2022-08-01", "2024-12-31");
INSERT INTO person VALUES (NULL, "Ahmed", "Bakir", 2000, "2022-08-01", "2024-12-31");
```

1.
```
SELECT * FROM person;

UPDATE person
FOR PORTION OF valid_time FROM "2022-11-01" TO "2024-12-31"
SET salary = 2500
WHERE id=1;
```
2.
```
SELECT * FROM person;
```
3.
```
SELECT * FROM person WHERE NOW() BETWEEN startdate AND enddate;
```
4.
```
SELECT * FROM person WHERE TIMESTAMP("2022-11-07") BETWEEN startdate AND enddate;
```

1.

id	firstname	lastname	salary	startdate	enddate
1	Viivi	Virta	2 000,00	2022-08-01	2024-12-31
2	Ahmed	Bakir	2 000,00	2022-08-01	2024-12-31

2.

id	firstname	lastname	salary	startdate	enddate
1	Viivi	Virta	2 500,00	2022-11-01	2024-12-31
2	Ahmed	Bakir	2 000,00	2022-08-01	2024-12-31
3	Viivi	Virta	2 000,00	2022-08-01	2022-11-01

3.

id	firstname	lastname	salary	startdate	enddate
2	Ahmed	Bakir	2 000,00	2022-08-01	2024-12-31
3	Viivi	Virta	2 000,00	2022-08-01	2022-11-01

4.

id	firstname	lastname	salary	startdate	enddate
1	Viivi	Virta	2 500,00	2022-11-01	2024-12-31
2	Ahmed	Bakir	2 000,00	2022-08-01	2024-12-31

Kyselyt kelpoisuusajan mukaan

- SELECT-kyselyt kirjoitetaan tavalliseen tapaan, ja aikapoiminta kirjoitetaan tarvittaessa WHERE-osaan.
- UPDATE- ja DELETE-lauseisiin voidaan liittää FOR PORTION -määre, joka muuttaa dataa annetulta aikaväliltä. Tauluun voi tämän seurauksena tulla uusia versioita tietueesta.
  - Edellisen esimerkin UPDATE-lause loi uuden aikavälin, jolla korkeampi palkka on voimassa. Vanha matalamman palkan aikaväli kutistui automaattisesti.
  - Huomaa, että UPDATE-lause loi uuden tietueen person-tauluun uudella perusavaimen arvolla. Jos id tulkitaan kiinteäksi henkilönumeroksi eikä tietuetunnisteeksi, tulee siitä tehdä tavallinen kenttä (ei perusavain).

Bitemporaalinen taulu

```
DROP TABLE IF EXISTS person;

CREATE TABLE person (
   id INT NOT NULL AUTO_INCREMENT,
   firstname VARCHAR(40),
   lastname VARCHAR(40),
   salary DECIMAL(8,2),
   startdate DATE,
   enddate DATE,
   PERIOD FOR valid_time(startdate, enddate),
   PRIMARY KEY (id)
) WITH SYSTEM VERSIONING;

INSERT INTO person VALUES (NULL, "Viivi", "Virta", 2000, "2022-08-01", "2024-12-31");
INSERT INTO person VALUES (NULL, "Ahmed", "Bakir", 2000, "2022-08-01", "2024-12-31");

SELECT *, row_start, row_end FROM person;
```

- Molemmat aikaulottuvuudet (kirjausaika ja kelpoisuusaika) voidaan ottaa samanaikaisesti käyttöön.

id	firstname	lastname	salary	startdate	enddate	row_start	row_end
1	Viivi	Virta	2 000,00	2022-08-01	2024-12-31	2022-08-12 14:05:37.071090	2038-01-19 05:14:07.999999
2	Ahmed	Bakir	2 000,00	2022-08-01	2024-12-31	2022-08-12 14:05:37.077544	2038-01-19 05:14:07.999999

Toinen vaihtoehto: mallinnus skeeman kautta

- Edellä käsitettiin tietokantaohjelmiston (MariaDB) temporaaliominaisuuksia.
- Olisiko vastaava toiminnallisuus mahdollista toteuttaa ilman niitä?
- Tarpeelliset historiataulut ja -kentät voidaan huomioida tietokantaskeemassa.
  - Transaktioaika saadaan SQL-kielen NOW()-funktion avulla.
- Skeema kyllä syntyy osaavissa käsissä, mutta päivitys- ja hakuoperaatiot vaikeutuvat huomattavasti.

Toinen vaihtoehto: mallinnus skeeman kautta

```
CREATE TABLE person (
   id INT NOT NULL AUTO_INCREMENT,
   firstname VARCHAR(40),
   lastname VARCHAR(40),
   PRIMARY KEY (id)
);

CREATE TABLE salary (
   salaryid INT NOT NULL AUTO_INCREMENT,
   personid INT,
   transactiontime TIMESTAMP DEFAULT NOW(),
   salary DECIMAL(8,2),
   startdate DATE,
   enddate DATE,
   PRIMARY KEY (salaryid),
   FOREIGN KEY (personid) REFERENCES person(id)
);
```

Toinen vaihtoehto: mallinnus skeeman kautta

- Kyselyistä ja päivitysoperaatioista tulee hankalampia.
- Kasvatetaan Viivin palkkaa tietystä päivästä lukien.
  - Tämä toimenpide menee jo melko monimutkaiseksi...
  - ... ja vastaavat asiat vaikeutuvat entisestään, kun skeema kasvaa.

```
UPDATE salary SET enddate="2022-11-01"
WHERE personid=1
AND enddate>"2022-11-01"
AND startdate<="2024-12-31";
INSERT INTO salary(personid, salary, startdate, enddate)
VALUES(1, 2500, "2022-11-01", "2024-12-31");

SELECT * FROM salary;
```



salaryid	personid	transactiontime	salary	startdate	enddate
1	1	2022-08-12 14:49:02	2 000,00	2022-08-01	2022-11-01
2	1	2022-08-12 14:49:05	2 500,00	2022-11-01	2024-12-31

Mallinnus skeeman kautta: etuja ja haittoja

- Etuja:
  - Aikaulottuvuus on tehty näkyväksi suunnittelun lähtökohdaksi.
  - Toimii jokaisessa tiedonhallintaohjelmistossa.
- Haittoja:
  - Skeemasta tulee monimutkainen.
  - Haku- ja päivitysoperaatioista tulee erittäin monimutkaisia.
    - Niiden monimutkaista toteutusta siirtyy vääjäämättä ohjelmistokerroksen vastuulle.
    - Tehokkuus voi kärsiä.

Kevyt ratkaisu: loki

- Jos historiaan liittyvät haut ovat hyvin harvinaisia eikä niitä tarvita osana järjestelmän normaalia toimintaa, voi tietokantaoperaatioiden kirjaaminen lokiin olla riittävä ratkaisu.
- Esimerkiksi MariaDB:ssä voidaan kytkeä päälle kyselyloki (*general query log*). Tällöin jokainen suoritettava SQL-lause tallentuu tekstitiedostoon.
- Historiatieto on tällöin periaatteessa haettavissa, mutta se on monimutkainen poikkeusoperaatio eikä onnistu normaalein SQL-hakukyselyin.

Lopuksi

- Aikaulottuvuuden tarve on syytä analysoida datan mallinnusvaiheessa.
- Temporaalitietokannoissa vaarana on ylimallintaminen: onko historia tallennettava kaikesta mahdollisesti muuttuvasta tiedosta?
    - Voiko esimerkiksi Suomen kunnan nimi muuttua? Pitääkö kunnan vanha nimi voida hakea aiemmassa muodossaan?
- Tietokantaohjelmiston tarjoamia aikaominaisuuksia kannattaa hyödyntää.
  – Esim. MariaDB:n bitemporaalisten taulujen tuki.

Metropolia